This course has already ended.

Chapter 18.1: Types and implicit values

About this page:

Key questions How to write a generic code without repeating itself?

Topics Type parameters, polymorphic functions, implicit parameters, typeclass design pattern.

What Will I Do? Mostly read, but a few assignments are done, too.

Indicative difficulty level: Partly quite challenging.

Rough Estimate of Workload:? 2 hours.

Point Value: 50 S

Related Projects: Parametric (https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Parametric/).

Introduction

During the autumn course, we learned about the functions `max` and `maxBy`, which were quite convenient to find the ”best” item in a collection. What if you needed the second best item, or would you like to get a collection with some desired number of best items?

As a first thought, you might sort the collection and pick the best items. This can usually be done relatively fast, but with larger numbers of items, the solution gradually becomes weaker. The solution below is looking for the three best items in an unsorted `Buffer` and returns them to the new `Buffer`. Imagine for the time being that this is an effective solution to the problem.

```
import scala.collection.mutable.Buffer

object ThreeBest extends App {

  val v = Buffer.fill(100){
    scala.util.Random.nextInt(100)
  }

  def threeSmallest( itemCollection: Buffer[Int]) = {

    // SortedSet is a structure that automatically keeps its content in order
    var smallest = scala.collection.mutable.SortedSet[Int]()
    for (k <- itemCollection) {
      smallest += k
      smallest = smallest.take(3)
    }

    smallest.toBuffer
  }

  println(threeSmallest(v))
}
```

Yes! Seems to work. With a little effort, you can then implement a version for strings, Double, etc., and of course for different collections: Vector, Array, List ….

Or then not. Somehow it seems that this is where the code is repeated again and again. (see: DRY (https://plus.cs.aalto.fi/studio_2/k2022/yleista/sanasto/#term-dry)) However, there is a solution to the problem.

Polymorphic functions

Let's start with a clearly easier problem. Let's do a function that returns the middle element of the buffer (`Buffer`) regardless of the buffer type.

```
def getMiddle[T](myBuffer: Buffer[T]) : T = {
  val returnValue : T = myBuffer(myBuffer.length / 2)
  returnValue
}
```

This is a so-called *polymorphic function* having, in addition to the usual parameters, also: a *type parameter*. Type names are written in capital letters, and the type parameter is no exception. In our example, we chose the name of our type variable to be `T`.

The type parameter(s) are presented immediately after the method name, after which they can be used in the type of the function parameters and the return value as "real" types. In addition, the type parameter is also used within a function, as a type of variables. As type inference still works the above example can be made a bit shorter:

```
def getMiddle[T](myBuffer: Buffer[T]) : T = myBuffer(myBuffer.length / 2)
```

One might think that in the example above, the use of the type parameter was even slightly exaggerated. Because it is not needed within the function and different parameters do not need to be separated, the same thing could have been done with a wildcard notation _ as follows: Let's try.

```
// BUT THIS CODE DOES NOT DO THE SAME //
def getMiddle[_](myBuffer: Buffer[_]) = myBuffer(myBuffer.length / 2)
```

It compiles and finds the middle item.... **But the return type is now Any** which makes the function infinitely less usable. Clearly this was not a place for a wildcard. They are however useful in a number of situations.

Inferring and setting parameter values

The call of a polymorphic function does not necessarily differ from calling a normal function, if the compiler can determine the value of the type parameter. Thus, the above method could well be called even in these ways:

```
getMiddle( Buffer("cat", "dog", "puppy") )
getMiddle( Buffer(3, 2) )
```

In the first case, type is inferred to be `String`, in the latter case, `Int`.

It is not always possible to infer the type because there are no parameters. The method we used in the autumn course `Array.ofDim [T] (size: Int)` is a good example of this. In the case of such a method, the type parameter had to be explicitly given:

```
val table = Array.ofDim[Double](5)
```

We could have done this with our own function as well:

```
getMiddle[Int]( Buffer(3, 2) )
```

Limiting the type parameter

There may be many type parameters and they may be recursive. In the example below (which as such does not work yet), an attempt has been made to construct a function that checks if a desired item can be found from a desired location in the given collection.

```
def containsTheSame[T, C[T]](item: T, itemCollection: C[T], index: Int) = {
   ???
}
```

This time, there are two type parameters, and the latter one, `C[T]` represents an unknown type with its own type parameter. However, we have requested that it should be of the same type as the item we gave, i.e. `T`.

We could call the method in the following way:

```
containsTheSame( "kissa", Buffer("kana", "koira"), 0)
containsTheSame( 2.5, Vector(1.0, 2.0, 3.0), 2)
```

It seems we have the type signature figured out. But how would we go about building such a function? It would seem logical to just write:

```
itemCollection(index) == item
```

... but this creates a problem. The generic class `C[T]` does not necessarily have `apply` method for retrieving an item from the collection at a particular index value. Thus, we need some way of telling that we want to limit `C[T]` to collections that can be searched for by the index of the item. Of Scala's collection categories, the most suitable for the situation is probably `Seq [T]`, which describes any sequential data structure. For example, Vector, Buffer, List are all sub-types of Seq.

It is this subtype relationship that we want to express here and it is done, as follows:

```
def containsTheSame[T, C[T] <: Seq[T]](item: T, itemCollection: C[T], index: Int) = {
    itemCollection(index) == item
}
```

Here, the operator `<:` implies that the type described by `C[T]` must be a subtype of `Seq[T]`.

The immediate benefit of this limitation is that we can now be sure that type `C[T]` must have at least all the characteristics defined for type `Seq[T]`. One of these is the `apply` method we need. Now our code works.

```
 println( containsTheSame('i', "siili".toSeq, 2))
res0: Boolean = true

 println( containsTheSame('5', Vector(7, 2, 1), 2))
res1: Boolean = false
```

Restriction in the other direction

Sometimes the restriction must be done in the opposite direction. Then, you need an operator `>:` . In the example below, it is required that type `U` is a supertype of `T`. Therefore, the type of *collection* `Buffer[U]` is either the same as `Buffer[T]`` or something more general. In both cases, the item can certainly be stored in the collection.

```
def assign[T, U >: T](item: T, itemCollection: Buffer[U]) = itemCollection.append (item)

class Person(name: String)
class Admin(name: String) extends Person(name)

val whatever = Buffer[AnyRef]()
val people   = Buffer[Person]()
val admins   = Buffer[Admin]()

assign(new Admin("Lauri"),  whatever)
assign(new Person("Esko"), whatever)

assign(new Admin("Lauri"),  people)
assign(new Person("Esko"), people)

assign(new Admin("Lauri"),  admins)

// The off commented code below code does not work because the parameter conditions are not met.
// This is not a programming error, but just right - no ordinary people should be
// saved into Admin Buffer and this could not be done outside the method either.

// assign(new Person("Esko"), admins)
```

Example - trial no. 2

Let's try to apply the ideas we have learned to the problem about the three smallest items given at the beginning of the chapter. Selecting the collection type to be `Iterable` covers almost all collection types. `Iterable` collections can be iterated, for example, with a for loop.

```
def threeSmallest2( itemCollection: Iterable[Int]) = {
  var smallest = scala.collection.mutable.SortedSet[Int]()
  for (k <- itemCollection) {
    smallest += k
    smallest = smallest.take(3)
  }

  smallest.toBuffer
}
```

That works with integers - but what if we turn the Int intoa type parameter?

```
def threeSmallest2[T]( itemCollection: Iterable[T]) = {
  var smallest = scala.collection.mutable.SortedSet[T]()
  for (k <- itemCollection) {
    smallest += k
    smallest = smallest.take(3)
  }

  smallest.toBuffer
}
```

Looks nice, but gives us the following compilation error:

line 2: error: No implicit Ordering defined for T.

The error message tells us that the SortedSet-class needs the items to be sortable somehow. More specifically, it needs an Ordering to work with. We don't know how to sort items of generic type T. And what does **implicit** mean?

Ordered and Ordering

It seems scala wants us to establish some order. We have two traits just for that: Ordered[T] and Ordering[T].

The trait `scala.math.Ordered[T]` describes everything that is comparable to an item that has the type specified by the type parameter. In practice it assures that the items with this trait have methods (and operators) like <, >=, etc. If we come up with anything we later want to sort, it's often enough to implement this trait. It only requires a single compare-function to be implemented.

```
class Person(age: Int) extends Ordered[Person]{
    def compare(other: Person) = this.age - other.age
}
```

`scala.math.Ordering[T]` is a class with methods for comparing two items of type T. The difference to Ordered is that Ordered is a feature of the items, whereas Ordering is implemented outside the class. This allows us to create a number of different orderings and even provide them for classes we cannot modify.

Let's take a look at the class `SortedSet`, or more precisely its companion object (https://www.scala-lang.org/api/current/scala/collection/SortedSet.html).

```
def apply[A](elems: A*)(implicit ord: Ordering[A]): SortedSet[A]
```

The first list contains the items we want to handle. `A*` means a variable length list of parameters. But what is going on in the second parameter list `implicit ord: Ordering [A]`? Clearly the apply method receives an Ordering-instance, which is needed by the SortedSet with the desired order.

`Implicit`

The parameters marked with **implicit** will automatically receive their value during the compilation time from the correspondingly defined variable, which is also marked with **implicit**.

In this case, `SortedSet` can be created with any type `T` and the implicit paramter makes sure that the compiler finds a corresponding Ordering[T] object that matches the desired type.

If no implicit value is found, or if we want to select it ourselves, the parameter can always be given "explicitly". For integers, the appropriate object is `scala.math.Ordering.Int`. In the example below, the order of items is reversed from the largest to the smallest by requesting a `reverse`-

version of the `Ordering` object.

```
import scala.collection.mutable.SortedSet
SortedSet(1, 2, 3)(Ordering.Int.reverse)
```

We can try writing a function with an implicit parameter ourselves. The code below will consider two items that match the `Ordered` trait and return the one, which is later in an alphabetical order. The type of the items is T, but a functionf is provided implicitly that can turn items of type T to type Ordered[T]

```
def better[T](first: T, second: T)(implicit f:(T => Ordered[T])) = {
   if (first > second)
     first
   else
     second
}
```

```
 better( "Flower", "Banana" )
res: String = Flower
```

Handy!

Typeclass design pattern

Typeclass is a design pattern from Haskell programming language such that can be used to add new functionality to types afterwards without changing the original classes. When the desired functionality is required for class X, the object implementing the functionality for that type is introduced. The types for which the entity implementing the functionality exists are considered belonging to the type class of this functionality. Type class is not a class, however, but rather a collection of classes that share the same feature.

`Ordering` is a good example of a type class. By adding the `Ordering` implementation to your class, class objects can be used almost everywhere in the Scala API where the organizing feature is needed. By saving our `Ordering` implementation to the `implicit` variable, the functionality is enabled automatically.

```
// Class where we want to sort objects
case class Person(name: String)

// We create Ordering[Person], and assign this to an implicit-variable
implicit val personOrdering =  new Ordering[Person] {
  def compare(a: Person, b: Person) = a.name.compare(b.name)
}

// Now SortedSet can "automatically" manage Person objects.
val set = SortedSet(Person("Matti"), Person("Anssi"))
```

Example, trial no. 3

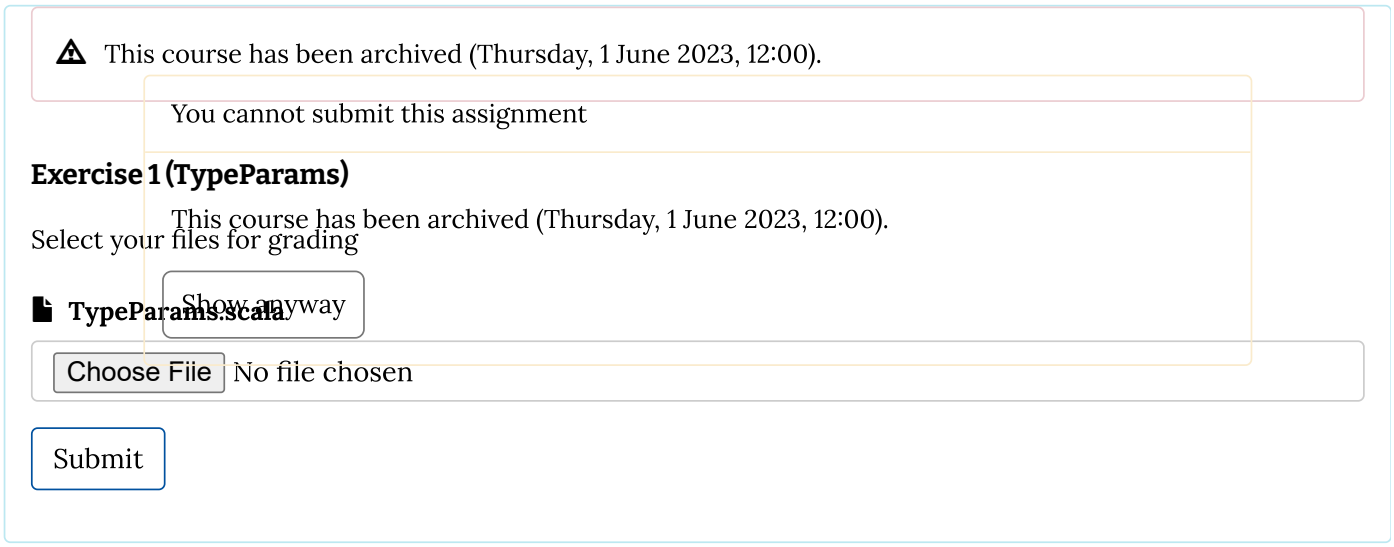
Our third attempt to make a general solution to finding the three smallest items is to use the techniques we learned with `SortedSet`. By adding the implicit parameter `ord` to the method, the compiler provides our method (and also the `SortedSet` constructor) with the `Ordering` object that is suitable for this method call. Now the code works on all collection types and all item types!

```
def threeSmallest3[T]( itemCollection: Iterable[T])(implicit ord: Ordering[T]) = {
  var smallest = scala.collection.mutable.SortedSet[T]()
  for (k <- itemCollection) {
    smallest += k
    smallest = smallest.take(3)
  }

  smallest.toBuffer
}
```

Exercise

There are four small functions in the Parametric class `parametric/TypeParams.scala` that use type parameters. These functions are implemented in the task. More detailed instructions can be found in the file TypeParams.scala.



Summary

- We familiarized with type parameters and polymorphic functions, and how to generate generic code with them.
- We presented the idea of implicit variable/parameter and how it is often applied in Scala API.

Many of the things you saw can be complex to understand and may have little use in the beginning. However, seeing them will hopefully clarify browsing the Scala API.

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

Loading assignment...